

Controlling the steering wheel using Deep Reinforcement Learning: a Survey

Narayana Darapaneni^{1*}, Anwesh Reddy Paduri^{2*}, Sudha BG^{1*},
Vidyadhar Bendre³, Midhun Chandran³, Mohana Priya M³,
Varghese Jacob³

^{1*}Director - AIML, PES University, Hosur Rd, Konappana Agrahara,
Bangalore, 560100, Karnataka, India.

^{2*}Senior Data Scientist, PES University, Hosur Rd, Konappana
Agrahara, Bangalore, 560100, Karnataka, India.

³Student-AIML, PES University, Hosur Rd, Konappana Agrahara,
Bangalore, 560100, Karnataka, India.

*Corresponding author(s). E-mail(s): darapaneni@gmail.com;
anwesh@greatlearning.in; sudha.m@gmail.com;
Contributing authors: vidyadhar.bendre@gmail.com;
midhunch@gmail.com; mohana739@gmail.com;
varghese.jacob4991@gmail.com;

Abstract

The increasing popularity of Autonomous Vehicles (AVs) has led to a growing interest in the development of advanced control algorithms to improve the safety and efficiency of these vehicles. This survey paper aims to provide an in-depth examination of the recent developments in the field of Reinforcement Learning (RL) for steering control in AVs. RL, a type of machine learning that focuses on training agents to make decisions based on trial-and-error, has emerged as a promising approach for steering control in AVs. This paper begins by providing an overview of the different RL algorithms that have been proposed for steering control in AVs, such as Q-learning, Deep Q-Networks, Double Deep Q-Networks, and Dueling Double deep Q-Networks. The paper then goes on to discuss the key challenges that need to be addressed in the application of RL for steering control. It also covers a brief comparison of RL algorithms Q-learning, Deep Q-Networks, Double Deep Q-Networks, and Dueling Double deep Q-Networks. Overall, this survey paper provides a comprehensive examination of the comparison in RL-based steering control algorithms and aims to provide

a valuable resource for researchers, engineers, and practitioners working in the field of autonomous driving for control task.

Keywords: Reinforcement Learning, Steering Wheel Control, Reinforcement Learning
A Survey of algorithms

1 Introduction

Reinforcement learning is a machine learning technique that enables an agent to learn by trial and error through interacting with its environment. The agent receives feedback in the form of rewards for positive outcomes and penalties for negative ones.

The main objective of the agent is to learn a policy, which maps states of the environment to actions, in order to maximize the cumulative reward over time. RL is applied to solve a much wider range of problems, including robotics, control systems, and gaming[1] [2].

The origins of RL can be traced back to the 1950s and 1960s, with early work by researchers such as Widrow and Hoff, who developed the "ADALINE" and "PERCEPTRON" learning rules. However, it wasn't until the 1980s and 1990s that RL began to be studied extensively. In 1989, Sutton and Barto published the first edition of their book "Reinforcement Learning: An Introduction" which is still widely used as a textbook in the field today. This book introduced many of the fundamental concepts and algorithms of RL, such as Q-learning and SARSA, and laid the foundation for much of the subsequent research in the field.

Q-learning is a well-known RL algorithm that learns an action-value function, in a given state, it estimates the expected cumulative reward for taking an action and following the learned policy thereafter. SARSA is a similar algorithm, but instead of using the learned policy to determine the next action, it uses the current policy. Both Q-learning and SARSA are off-policy algorithms, which means that they can learn about the value of actions that are not currently being taken. In the 2000s, RL saw increased interest and many new algorithms were developed. One important development was the actor-critic method, which separates the learning of the policy and the value function. The actor learns the policy, while the critic learns the value function. This allows for more efficient learning and better handling of continuous action spaces. Another important development was deep RL, which combines RL with deep neural networks. This allows RL to be applied to problems with large state and action spaces and has led to the development of many successful deep RL algorithms, such as DQN and A3C. Additionally, evolutionary methods were also incorporated in RL to improve the stability and sample efficiency of the algorithm[3] [4].

In recent years, RL has made significant progress, especially in the area of deep RL. Deep RL algorithms like DQN, A3C and DDPG, PPO, TRPO, SAC, TD3, D4PG applied to different tasks, like playing Atari games, controlling robots, and optimizing control systems. For example, in 2015, a team of researchers at Google DeepMind used a variant of the DQN algorithm called the Asynchronous Advantage Actor-Critic (A3C) to train an agent to play a wide range of Atari games at a superhuman level.

This was a major breakthrough in the field, as it showed that RL could be used to solve complex, high-dimensional problems[3] [4].

Recently, RL is applied by combining with other techniques like imitation learning, meta-learning, and inverse RL to improve its sample efficiency and stability. In Imitation learning (IL) an agent learns to imitate the behavior of a demonstration provided by an expert. Meta-learning is a technique in which an agent learns to learn, by adapting quickly to new tasks. Inverse RL is a technique that tries to infer the reward function from an agent’s behavior. These techniques have shown to be effective in improving the sample efficiency and stability of RL algorithm[5] [6].

2 LITERATURE REVIEW:

Reinforcement learning is a machine learning technique that enables an agent to learn by trial and error through interacting with its environment.

The origins of RL can be traced back to the 1950s and 1960s, with early work by researchers such as Widrow and Hoff, who developed the "ADALINE" and "PERCEPTRON" learning rules. However, it wasn’t until the 1980s and 1990s that RL began to be studied extensively, with the development of Q-learning and SARSA, two popular RL algorithms.

In the 2000s, RL saw increased interest and many new algorithms were developed, including actor-critic methods, deep RL, and evolutionary methods. RL is applied to solve a much wider range of problems, including robotics, control systems, recommender system and gaming.

In recent years, RL has made significant progress, especially in the area of deep RL. Deep RL algorithms like DQN, A3C and DDPG, PPO, TRPO, SAC, TD3, D4PG have been successfully applied to different tasks, like playing Atari games, controlling robots, and optimizing control systems[4]. One promising application is in the field of autonomous vehicles.

We initially investigate the performance of these algorithms in the widely studied Cart-Pole environment, showcasing their ability to learn complex control policies. Building on these results, we discuss how the key concepts and strategies developed for Cart-Pole can be adapted and extended to the more complex task of controlling a steering wheel in real-world driving scenarios. By drawing parallels between the simplified Cart-Pole environment and steering control challenges, this study aims to bridge the gap between theoretical DRL research and practical applications in autonomous vehicles, providing valuable insights for researchers and engineers working in this domain.

It is worth mentioning that RL has not been without its challenges, including the problem of sample inefficiency, instability, and the difficulty of handling large state and action spaces. However, despite these challenges, RL continues to be an active area of research with many promising applications in the field of autonomous vehicles. More recently, RL has been combined with other techniques like imitation learning, meta-learning, and inverse RL to improve its sample efficiency and stability.

3 MATERIALS AND METHODS

3.1 Dataset and Environment:

For this experiment, we have used openAI gym environments and the associated data. The Q-Learning/Q-NETWORK ALGORITHM IS IMPLEMENTED AND TESTED USING TAXI-V3 environment.

The rest three algorithms (DQN, DDQN and Dueling-DDQN) is implemented and assessed using Cartpole-v1 environment. The state and action space details of the environment used in this paper are available in the OpenAI gym official documentation. [7]

3.2 Core Reinforcement Learning Algorithms

- Q-Networks: Q-Networks (also known as Q-Learning) are a type of model-free, off-policy reinforcement learning algorithm that can be used to learn control policies for autonomous systems.

In Q-Learning, the agent learns a function called the Q-function, for each possible action in each state of the environment it estimates the expected return (i.e., the sum of future rewards). Based on the agent's current understanding of the environment, the Q-function is used to select actions that are likely to lead high returns.

The Q-Network is used to learn a policy. The policy enables the vehicle to navigate safely and efficiently through traffic. The Q-Network is trained using a dataset of driving scenarios, in which the agent receives rewards for making safe and efficient driving decisions.

Overall, Q-Networks can be a useful tool for learning control policies for autonomous systems, including steering control in autonomous driving. However, it is crucial to design the reward function and the training process to achieve good performance.

The mathematical formula for Q-learning could be mapped to this context:

$$Q(s, a) = Q(s, a) + \alpha[r + \gamma \max_a(Q(s, a)) - Q(s, a)] \quad (1)$$

$Q(s, a)$: This term represents the current estimate of the Q-function for state s and action a . The state s represents the current location and orientation of the vehicle, as well as various other factors such as the speed and acceleration of the vehicle, the road conditions, and the presence of other vehicles or pedestrians. Action a represents the steering angle of the vehicle or a more complex action that includes multiple components such as the steering angle, acceleration, and braking.

$Q(s', a')$: This term represents the maximum expected future reward for all actions a' that can be taken in the next state s' . The state s' represents the state of the vehicle after taking action a' in state s , and the actions a' represent the possible actions that can be taken from state s' . The expected future reward would depend on the performance of the vehicle in terms of safety, fuel efficiency, and other factors.

r : This term represents the immediate reward received for taking an action a , in state s . The immediate reward is based on the distance travelled the fuel efficiency of the vehicle, and any penalties incurred for collisions or other unsafe behaviours.

γ : This is the discount factor, which determines the importance of future rewards relative to the current reward. The discount factor is set to a higher value if the long-term

performance of the vehicle is more important than the immediate reward, or a lower value if the immediate reward is more important.

α : This is the learning rate, which determines the degree to which the new estimate of the Q-function is influenced by the old estimate and the difference between the old estimate and the target value. The learning rate is set to a higher value if the agent is learning from scratch, or a lower value if the agent has already learned a good policy and only needs to make small adjustments.

Using the above formula, the Q-learning algorithm would iteratively update the Q-function estimates for the steering wheel control task, based on the immediate rewards received and the expected future rewards for different actions in different states. The process would continue until the Q-function converges to the optimal action-value function, at which point the agent would have learned the optimal policy for selecting actions that lead to safe and efficient navigation

- Deep Q-Network: DQN (Deep Q-Network) is an extension of the Q-Learning algorithm that uses a neural network to approximate the Q-function. Like Q-Learning, DQN can be applied to the problem of controlling the steering wheel in autonomous driving. In DQN, the Q-function is represented as a neural network, which takes as input the current state of the environment and outputs a prediction of the expected return for each possible action. The neural network is trained using a dataset of driving scenarios, in which the agent receives rewards for making safe and efficient driving decisions [8] [9]. A DQN is used to learn a policy that enables the vehicle to navigate safely and efficiently through traffic. The DQN is trained using a dataset of driving scenarios, in which the agent receives rewards for making safe and efficient driving decisions [4] [6].
- Deep Double Q-Networks (DDQN): Deep Double Q-Networks (DDQN) is an extension of the Q-learning algorithm. DDQN is a model-free reinforcement learning method for solving MDPs (Markov decision processes). DDQN is used to learn a policy for determining the optimal actions to take at each state to maximize a reward signal.

The DDQN algorithm works by estimating the action-value function, DDQN is a measure of the expected future reward for taking a particular action at a given state. The algorithm uses a neural network to approximate this function and updates the network's weights through an iterative process of training on a dataset of transitions experienced by the autonomous vehicle.

The "double" in DDQN uses two separate networks to estimate the action-value function: a "target" network and a "local" network. The target network is used to generate target values for training the local and is updated less frequently than the local network to stabilize the learning process.

To apply DDQN to the problem of controlling a steering wheel in an autonomous driving scenario, the states could include variables such as the car's position, velocity, and heading, and the actions could include turning the steering wheel to the left, right, or keeping it straight. The reward signal is based on factors such as the smoothness of the car's movement, or the proximity of the car to other objects in the environment. The DDQN algorithm would then learn a policy for selecting actions at each state that maximizes the expected reward [10].

- **Duelling Deep Q-Networks:** The Duelling Deep Q-Networks (Duelling DQN) is an extension of the Deep Q-Network (DQN) algorithm. It is a model-free reinforcement learning method for solving Markov decision processes (MDPs). Like DQN, Duelling DQN uses a neural network to estimate by approximating the action-value function and learn a policy for selecting actions at each state.

One key difference between DQN and Duelling DQN is the way in which the action-value function is estimated. While DQN estimates the action-value function directly, Duelling DQN decomposes the action-value function into two separate functions:

- Advantage function, which estimates the relative value of each action
- Value function, which estimates the overall value of each state.

This decomposition allows Duelling DQN to learn more efficiently by separately representing the value of each action and state, rather than estimating the action-value function directly. To implement Duelling DQN, the neural network architecture is modified to have two separate streams:

one for advantage function and another for the value function. The output is then obtained by combining the outputs of the two streams using a special aggregation function.

Duelling DQN has been shown to be effective in multiple reinforcement learning tasks. Even in playing Atari games and controlling robots. It has also applied to multiple real-world problems. For example in recommendation systems, natural language processing, automated trading and autonomous driving.

4 RESULTS

- **Q-Networks:** Q-Learning starts with trial-and-error method to reach the destination. From the Fig 1, it is evident that it requires 1000 iterations to converge. The learning between iteration 200 to 1000 is very minimal. The code to q-learning algorithm is published in github[11].

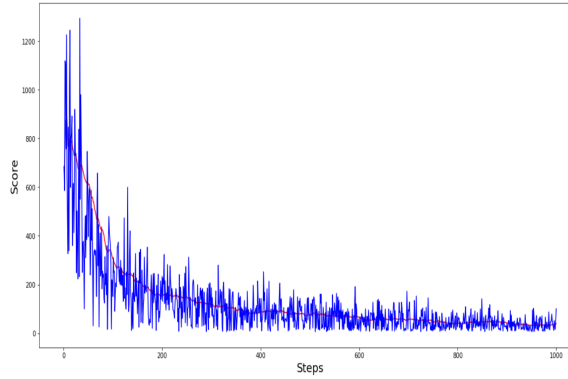


Fig. 1 Q-learning results

- Deep Q-Network: From the Fig 2 it is evident that, Deep Q-Learning can learn from smaller amounts of data as it uses a neural network to arrive at the Q-function by approximation. It uses techniques like experience replay and fixed Q-targets to improve stability and reduce sensitivity to hyperparameters.

The moving average is increasing even with minimal iterations as observed in the graph. 5 and 6. The code to Deep Q-Network algorithm is published in github[11].

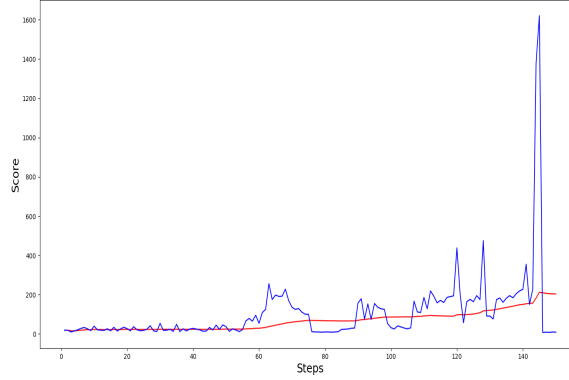


Fig. 2 Deep Q-network results

- Double Deep Q-Networks (DDQN): From the Fig 3, the presence of double Neural network has aided in improving the moving average faster than in Deep Q-Network. The code to DDQN algorithm is published in github[11].

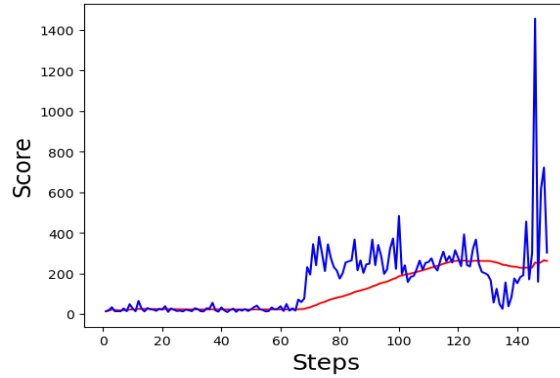


Fig. 3 Double Deep Q-network results

- Duelling Double Deep Q-Networks: From the Fig 4, it is evident that there is a steep increase in the score and the moving average due to advantage and Q-value function. This method is performing better than prior methods. The code to D3QN algorithm is published in github[11].

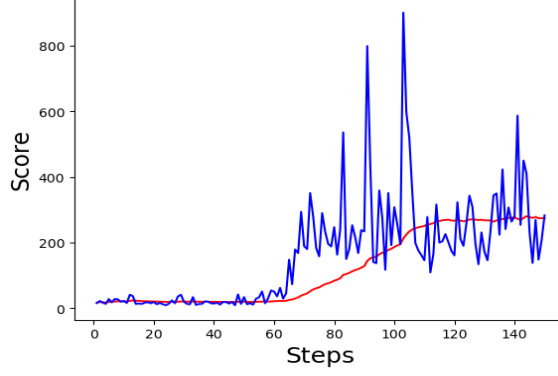


Fig. 4 Duelling Deep Q-network results

5 CONCLUSIONS

While Q-Learning, DQN, DDQN, and D3QN are all reinforcement learning algorithms that use value-based methods to solve Markov Decision Processes (MDPs), they differ in their approach to estimating the action-value function.

Q-Learning is a reinforcement learning algorithm that uses a table to store the action-value function. It can be unstable and is often subject to the problem of overestimation and this can be seen clearly in the result as in Fig 1.

DQN (Deep Q-Network) is a deep reinforcement learning algorithm uses a neural network to estimate the action-value function. It addresses some of the limitations of Q-learning by using a fixed target network and experience replay. The improvement in the algorithm is shown in Fig 2.

DDQN (Double Deep Q-Network) is an extension to DQN that addresses the problem of overestimation of action values. It uses two separate neural-networks(NNs) to estimate the action-value function and improves stability by using the primary network to choose actions and the target network to evaluate them. The improvement in the algorithm can be seen in the Fig 3.

D3QN (Duelling Double Deep Q-Network) is an extension to DDQN. It separates the estimation of the value function and the advantage function, allowing for a more efficient estimation of the action-value function. The improvement in the algorithm can be seen in the Fig 4.

In summary, DQN, DDQN, and D3QN are all improvements over the basic Q-Learning algorithm, offering better stability and performance using neural networks and other techniques. D3QN is the most advanced of these algorithms and offers the best performance, but also requires more computational resources. From Fig 5 and 6 it is evident that D3QN outperforms all other algorithms even with less episodes. The Q Networks requires more training episodes as opposed to all other algorithms; it could be because Q-networks rely on trial and error methods to populate the Q-Table. D3QN has better stability against sub-optimal convergence.

Future Scope: In this survey we have done comparison on only 4 algorithms

	Average qn -ve scores	Averages dqn	Averages ddqn	Averages d3qn
Episodes				
1	684.00	18.00	11.00	32.00
2	635.00	16.50	14.00	22.50
3	796.00	15.00	21.33	22.00
4	874.25	15.50	19.75	20.25
5	850.80	17.80	22.40	22.00
...	...	...	...	...
146	212.00	141.50	139.92	318.52
147	214.70	140.20	137.62	319.74
148	214.84	138.64	137.36	318.94
149	213.78	138.38	137.46	315.04
150	212.36	137.10	134.88	312.60

150 rows x 4 columns

Fig. 5 Moving Average Comparison Table

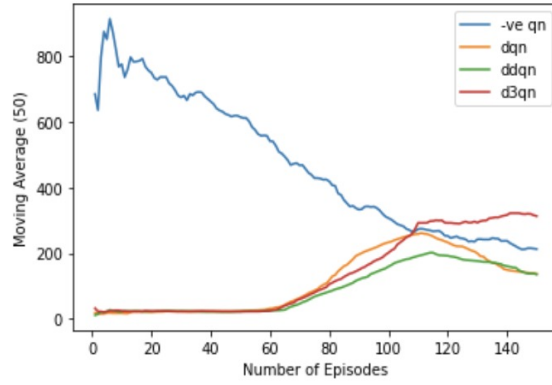


Fig. 6 Moving Average Comparison Graph

1. Q-Networks
2. Deep Q-Networks
3. Double Deep Q-Networks
4. Duelling Double Deep Q-Networks

We can extend the study to compare all the algorithms used in steering wheel control that are explained in this paper. We have fixed the number of episodes as 150 because it is computationally heavy and as a future study one can consider a larger number for the number of episodes. We have not considered the impact of algorithms when they are combined to cancel out each-others shortcomings. As an extension to this study one can choose different algorithm combining techniques such as

- **Model Combination:** The model of Duelling DDQN can be combined with other DRL models, such as Soft Actor-Critic (SAC), to create a new model that leverages the strengths of both algorithms. The combined model can benefit from the separate representation of the Q-function in Duelling DDQN and the entropy-regularized reinforcement learning in SAC. [3]

- **Technique Combination:** The techniques used by Duelling DDQN can be combined with other techniques, such as Prioritized Experience Replay, to improve its performance. Prioritized Experience Replay can address the slow convergence problem in Duelling DDQN by sampling transitions more frequently from the replay buffer based on their temporal difference errors.
- **Output Combination:** The outputs of Duelling DDQN can be combined with other outputs, such as the policy estimated by Trust Region Policy Optimization (TRPO), to create a new policy that leverages the strengths of both algorithms. The new policy can benefit from the robust and stable estimates of the Q-function in Duelling DDQN and the high-dimensional policy optimization in TRPO.

For those interested in reviewing the source code, it can be accessed via github[11]. This will provide valuable insights into the implementation details and underlying algorithms discussed in this paper.

References

- [1] Michalík, R., Janota, A.: The pybullet module-based approach to control the collaborative yumi robot. In: 2020 ELEKTRO, pp. 1–4 (2020). <https://doi.org/10.1109/ELEKTRO49696.2020.9130233>
- [2] Elallid, B.B., Benamar, N., Hafid, A.S., Rachidi, T., Mrani, N.: A comprehensive survey on the application of deep and reinforcement learning approaches in autonomous driving. *Journal of King Saud University-Computer and Information Sciences* (2022)
- [3] Ultsch, J., Mirwald, J., Brembeck, J., Castro, R.: Reinforcement learning-based path following control for a vehicle with variable delay in the drivetrain. In: 2020 IEEE Intelligent Vehicles Symposium (IV), pp. 532–539 (2020). <https://doi.org/10.1109/IV47402.2020.9304578>
- [4] Emuna, R., Borowsky, A., Biess, A.: Deep reinforcement learning for human-like driving policies in collision avoidance tasks of self-driving cars. Workingpaper (2020)
- [5] Osiński, B., Jakubowski, A., Ziecina, P., Miłoś, P., Galias, C., Homoceanu, S., Michalewski, H.: Simulation-based reinforcement learning for real-world autonomous driving. In: 2020 IEEE International Conference on Robotics and Automation (ICRA), pp. 6411–6418 (2020). <https://doi.org/10.1109/ICRA40945.2020.9196730>
- [6] Zhang, J., Chen, H., Song, S., Hu, F.: Reinforcement learning-based motion planning for automatic parking system. *IEEE Access* **8**, 154485–154501 (2020) <https://doi.org/10.1109/ACCESS.2020.3017770>
- [7] Wu, K., Abolfazli Esfahani, M., Yuan, S., Wang, H.: Learn to steer through deep reinforcement learning. *Sensors* **18**(11), 3650 (2018)

- [8] Pérez-Gil, Ó., Barea, R., López-Guillén, E., Bergasa, L.M., Gómez-Huélamo, C., Gutiérrez, R., Díaz-Díaz, A.: Deep reinforcement learning based control for autonomous vehicles in carla. *Multimedia Tools and Applications* **81**(3), 3553–3576 (2022)
- [9] Maramotti, P., Capasso, A.P., Bacchiani, G., Broggi, A.: Tackling real-world autonomous driving using deep reinforcement learning. In: *2022 IEEE Intelligent Vehicles Symposium (IV)*, pp. 1274–1281 (2022). IEEE
- [10] Kiran, B.R., Sobh, I., Talpaert, V., Mannion, P., Al Sallab, A.A., Yogamani, S., Pérez, P.: Deep reinforcement learning for autonomous driving: A survey. *IEEE Transactions on Intelligent Transportation Systems* (2021)
- [11] CodeBase by Vidyadhar Bendre, Mohana Priya M, Varghese Jacob, Midhun Chandran. https://github.com/midhuncnair/RL_SteeringWheelControlSurvey